第 10 章　用机器做一连串的加法

在前面我们造出了加法机，几样简单的东西——继电器、开关、灯泡，还有电线，就能做算术题，真是了不起。都说猴子是人类的祖先，而且还是活物，你要是问它，它也只能是朝你翻翻眼、龇龇牙，这就是回答。上中学的时候，我有一个同学不知怎么地把驴给惹毛了，弹了他一蹄子。如果他当时的意思是让驴算数学题，那么他额头上的“0”无疑就是驴给出的最佳答案。

和现代的计算机相比，用这台机器算加法，对人和机器都是一种考验。一方面，机器需要忙碌的运转才能算出结果；另一方面，人也没法消停，得在旁边侍候着，像保姆一样扳动开关，输入数据，然后等着结果出来。这还不算什么，最让人心存疑惑的是，用它来算加法还不如我们用纸和笔来得快，充其量也就是一个玩具，发明这东西有用吗？

当然是没有用——如果我们一直停留在这里原地不动的话。好在现实的情况并非如此。如果说现在你正在使用的计算机是一个智力健全的成年人，那么这台加法机就是一个刚出生的婴儿。从加法机到现代的计算机，这中间还有很多路要走，在这一章里，我们将继续向着这个宏伟的目标再迈出那么一小步。

10.1　把一大堆数加起来

传统上，“机”给我们的印象是，它们都是一些大家伙。相反，我们前面已经发明出来的、能做加法的东西就像汽车里的发动机一样，将被当成一个用于制造计算机的零部件，所以称之为“加法器”或者“加法部件”可能更符合它的身份。

名称不是大问题，不管叫什么，也掩盖不了它其实就是一大堆全加器的事实。像以前一样，用加法器来算数学题，需要用一排开关拼成一个被加数，比如 10011（十进制数 19）；用另一排开关拼成一个加数，比如 00101（十进制数 5），然后加法器就会自动算出结果 11000（十进制数 24），如图 10.1 所示。

注意，不像以前，在这里我们把加法器画成了两边高中间凹的桶形，这很形象地表明它要接收两路输入，相加后形成一路输出。如果这两排开关是我们头上的两只眼睛，那么加法器就是汇总和加工它们的大脑。当然，除了这一点比较相似之外，我们的脑袋并没有长成加法器那样的桶形，真是庆幸。

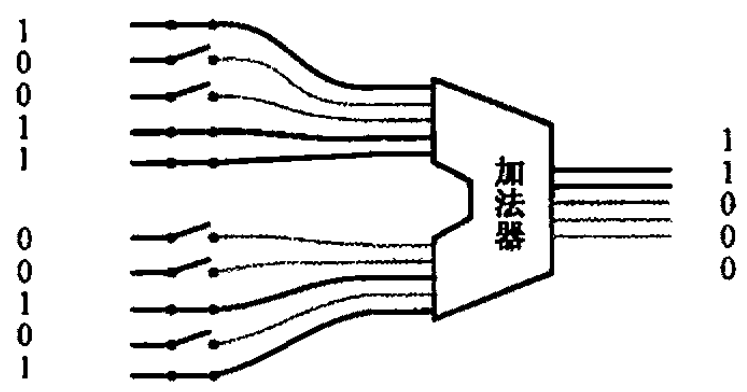

图 10.1　使用两排开关分别给出被加数和加数，加法器就会算出结果

只做一次运算，仅仅把两个数相加，这很简单。怕的是有好几个数相加，这可就热闹了。比如：

$$10+5+7+2+6$$

首先，你需要将 10 和 5 相加，也就是 01010 加上 00101，得出 01111，即十进制数 15。

01111 只是一个中间结果，因为它还要加上后面的数。所以，我们还得再次折腾那两排开关，分别把它们扳成 01111 和 00111，也就是上一步相加的结果 15，以及下一个要加的数 7。如果加法器工作正常，而且灯泡都没坏的话，结果肯定是 10110，也就是十进制数 22。

很显然，除了一开始，以后每次都有一个中间结果参与计算，都需要拨弄一番开关，将上一次算出来的结果再作为加数或者被加数输入到加法器里。

这当然是很不方便的。相信你肯定用过那种可以放在口袋里的袖珍计算器，相比之下，它就善解人意，从来不要求你记住每次相加的中间结果，也用不着把中间结果再输入一次。相反，你只需输入一个数，按一次“+”；再输入下一个数，再按一次“+”；……，就这样输入所有要相加的数即可得到结果。相比之下，我们的这个加法器在连续做加法时就显得有些笨拙了。

这里面的奥秘是什么呢？

其实，即使是袖珍计算器也不能在不使用中间结果的情况下计算数学题，只不过它不想让我们操心这些事情，所以就大包大揽。我们已经讲过移位寄存器，同时也跟你说过所谓的“寄存器”是什么意思。在袖珍计算器的内部，也有一个寄存器，可以保存一个二进制数，中间结果就保存在这里。然后，每当你输入另一个数时，它就被悄悄地安排来参与下一次计算，算出来的结果还放在这里。

保存一个二进制数？嗯，我想这里需要一个新的发明。我们知道，一个触发器只能保存 1 个比特，但是一个完整的二进制数通常包含好几个比特，是一个比特串。而且，二进制数中的所有比特都必须一起处理。

取决于想要保存的二进制数有多大，寄存器通常由好多个边沿 D 触发器共同组成。举个例子，如图 10.2 所示，这个寄存器包含了 5 个上升沿 D 触发器，所以能用来保存一个 5 比特长的二进制数。

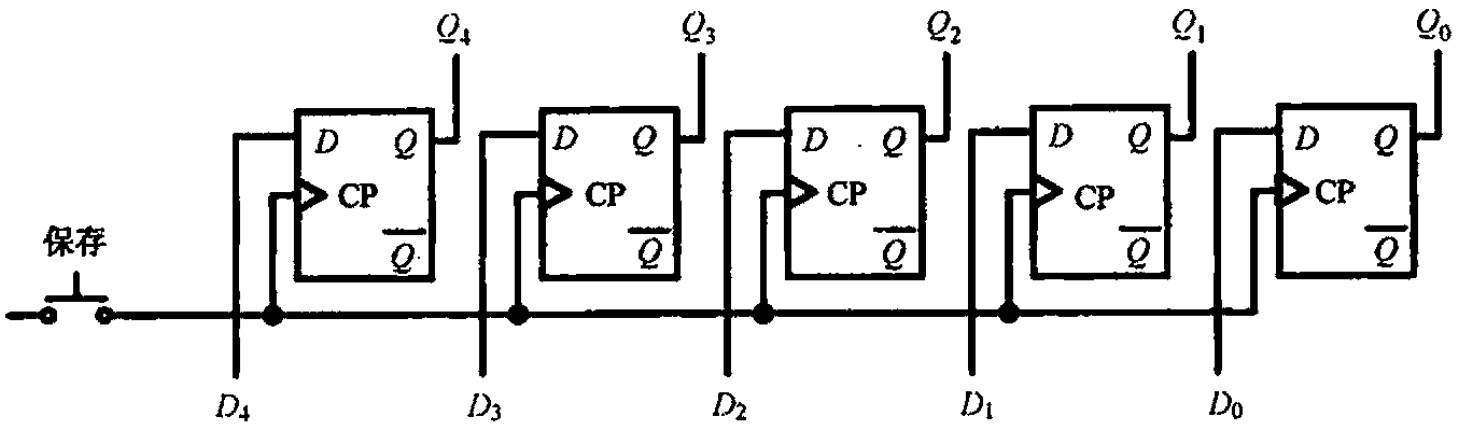

图 10.2　使用多个触发器可以构成一个寄存器

你明白了吗？我的意思是，不管一个二进制数包含多少个比特，要保存它，只需要把它的每一个比特都保存起来即可。所以，在这个例子中，被保存的二进制数，它的每一位，都分别进入 D_0……D_4 这 5 根线；同时，如你所见，所有触发器的 CP 端都连在一起，这样就可以接收同一个控制命令。一旦“保存”开关按下，在 CP 脉冲的上升沿，所有触发器同时开始干活，二进制数的每一位都在同一时间被保存起来，并立即出现在 Q_0……Q_4 上。

同样是为了方便，寄存器需要一个简明的图示，否则到了后面，当你摸不着头脑的时候又要怪我。寄存器的符号如图 10.3 所示，注意那个三角形，这表明该寄存器只在 CP 脉冲的上升沿才会工作。

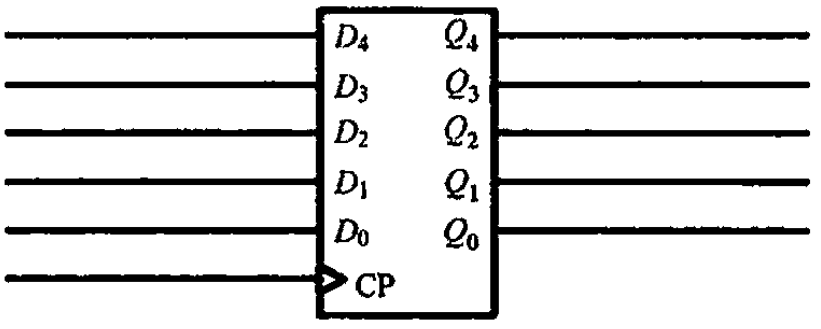

图 10.3　寄存器的符号

在伟大的寄存器先生隆重出场之后，做数学题的时候再也不用关心那些中间结果了。我们应该把它连到加法器上，越快越好。不过，把它放到哪个位置呢？

如图 10.4 所示，RA 就是我们刚刚造出的寄存器，它在加法器的前排就座。按键开关 K_{RA} 和 RA 的 CP 端相连，当我们用左边那一排开关扳出一个数之后，如果按一下 K_{RA}，这个数就被锁住。与此同时，它把自己吃进来的内容输送到加法器，并一直保持，作为第一个要相加的数。

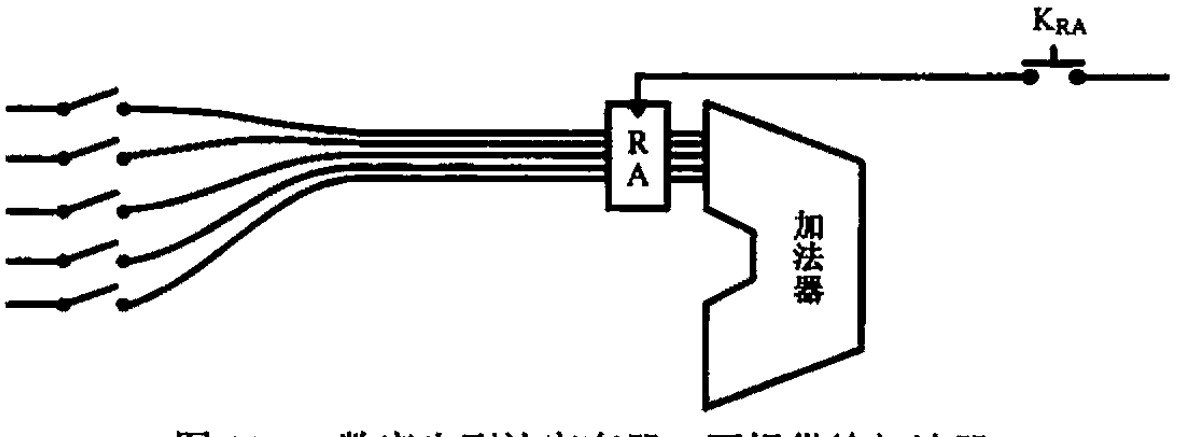

图 10.4　数字先到达寄存器，再提供给加法器

现在，我们想用同一排开关向加法器提供另一个数。之所以不像以前那样分别用两排开关来提供被加数和加数，是因为这样能使电路看起来更简单，也可能是附近商店里的开关都卖完了，所以不得不这样做。当然，我知道这样的理由很牵强，糊弄不了你，真正的原因到了后面我不说你也能自己领悟。总之，用一排开关也很不错，如图 10.5 所示。

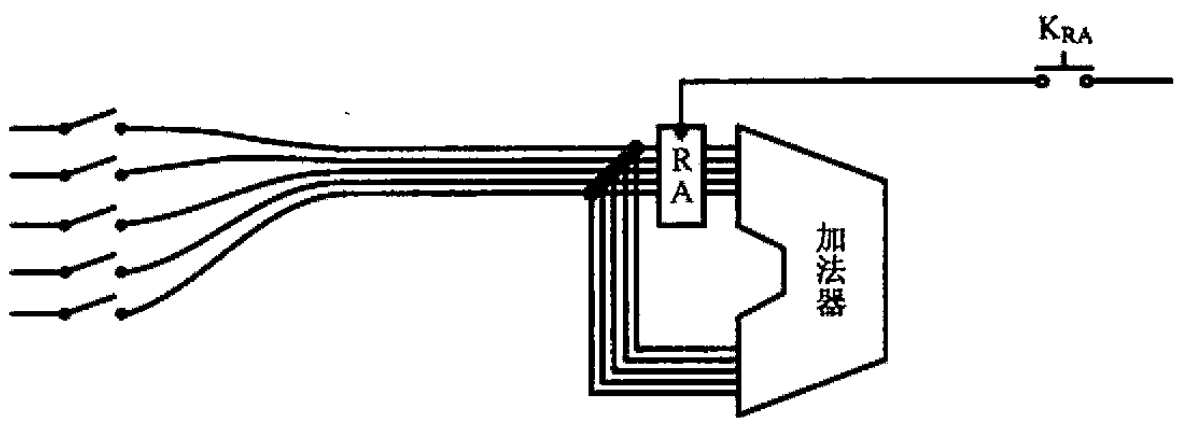

图 10.5　用同一排开关既提供被加数、也提供加数

经过改造之后，通过左边那排开关送进来的数可以到达寄存器 RA，同时也被送到加法器的另一个输入端。取决于你的动机，如果你想把它保存到寄存器 RA 中，那就按一下 K_{RA}；如果你想用它和 RA 中的数相加，就什么也不做，结果自然会从加法器的输出端呈现。

不得不在这里称赞一下加法器的工作态度和敬业精神。它很勤奋，随时都在自觉自愿地计算，不需要下达命令。当用开关摆出第二个数的同时，相加的结果也出来了（甚至就在你摆动每个开关的同时，它也在做计算，在这里表扬一下）。

要在往常，这个中间结果应该记下来，然后重新用开关输入运算器，和下一个数相加。但是，由于我们已经下决心甩开这个包袱，所以这个中间结果可以保存到寄存器 RA 中，这样就很自然地为下一次计算准备好了一个中间结果。

为了达到这个目的，一个可能的方案如图 10.6 所示，把加法器的输出同寄存器 RA 的输入端直接相连。

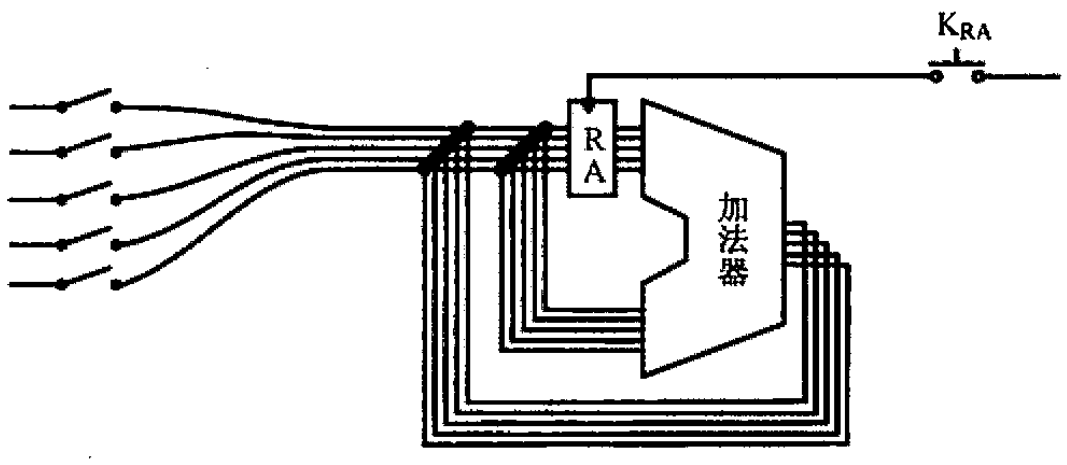

图 10.6　加法器的计算结果应当返回寄存器中

想法不错，不过这里有几个麻烦。首先，左边那排开关和加法器的输出是直

接相连的，都要走寄存器 RA 门前那段路。在逻辑电路里，大家共用的公共线路称为总线。想一想如果好几列火车都企图在同一时间通过一段铁轨时会发生什么，要是不把它们隔开，左边那排开关和加法器的输出都会抢着与寄存器 RA 说话。在这种情况下，就像刚刚从恋爱阶段过渡到琐碎生活的男女一样，在不走运的情况下这极有可能发生电气上的冲突，并产生神奇的烟火和气味。

其次，从图中可以看出，即使不考虑电路冲突，计算结果在从加法器出来之后，不但会送往寄存器 RA，还会再次进入自己的输入端，这理所当然地会形成一个反馈，而且是一边反馈、一边还在做加法，一切全乱了套。在这种情况下，你还能指望保存在寄存器 RA 中的数是正确的吗？

要彻底解决这个问题，就必须重新设计整个电路。一个最简单的解决方案是使用电子开关，更多的时候，我们也称之为传输门。

10.2 轮流使用总线

最简单、最好理解的电子开关就是一只我们再熟悉不过的继电器，通过控制继电器线圈中电流的有无，可以间接地控制另一个电路的通断。

二进制数的所有比特都是同时传输的，所以就需要好几个继电器来分别接通或者切断它的每一位。如图 10.7 所示，我们把所有继电器的线圈都接在同一个开关上，使它们同时吸合或者释放，就可以达到目的。

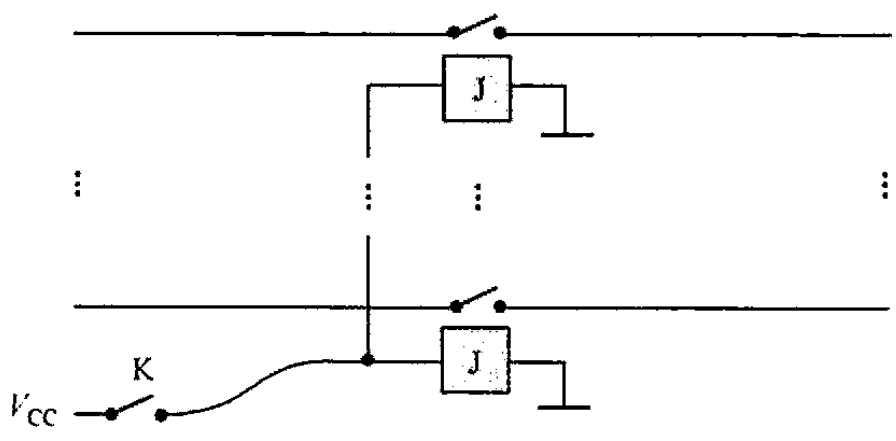

图 10.7 用一只开关来控制多条线路的通断

采用继电器来制造这个隔离装置，当然又直观又容易理解，但它并不是最好的材料，特别是在半导体技术出现之后。使用半导体材料，我们可以制作出更小、更便宜、更省电的这种电子开关来。哦，对了，“电子开关”并不是一个专业的称谓，它真正的名字应该是“传输门”。实际上，它确实像门，打开它，信号可以从一边传到另一边；关上它，就传不成了。

原则上，要通过总线传送数据的任何一方都应该使用传输门以免互相干扰。如图 10.8 所示，左边那排开关通过传输门 GA 接入总线；加法器的输出则通过另一个传输门 GB 接入总线。

除此之外，你可能已经发现该电路还多了一样东西。对了，确实多加了一样东西，这就是临时寄存器 TR，它和 RA 是一模一样的，不同之处在于，它要用来临时保存加法器的计算结果。

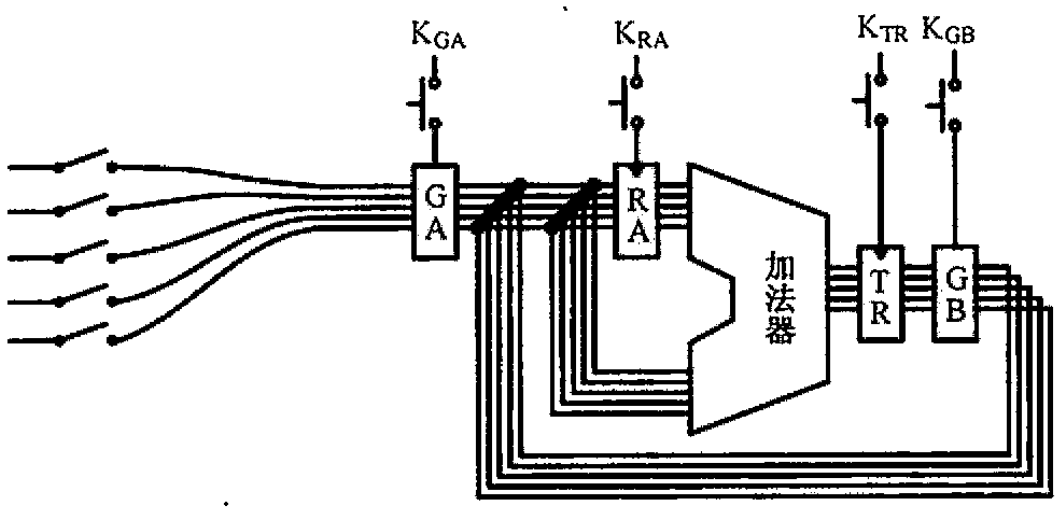

图 10.8　完整的加法运算电路，它可以将多个数字相加

用这个新设计的电路做加法是很有趣的（当然，可能有些烦琐）。考虑到你是个新手，我想我应该手把手地教你实际做一次，来看看它是如何工作的。在开工之前，K_{GA}、K_{GB}、K_{RA}、K_{TR} 都应当是断开的（我觉得强调这一点有些多余，因为它们都是按键开关，只要你不碰它们，它们就一直处于断开状态）。

在前面，我们采用的例子是计算

$$10+5+7+2+6$$

为此，我们首先要做的是用左边那排开关扳出第一个数“10”，并将其保存到寄存器 RA 中。万事开头难，这是开场动作，我们不妨称之为“装载”。

装载的过程是这样的：假设数已经扳好了，接下来，按住 K_{GA} 不要松开，使传输门 GA 打开，于是数据到达寄存器 RA；接着，再按一下 K_{RA} 将数据锁进 RA 中；最后，松开 K_{GA}。

当然了，从图 10.8 可以看出，这个数不但到达寄存器 RA，还到达加法器的另一个输入端，以及传输门 GB。同时，加法器也一直在工作。但是，因为传输门 GB 没有打开，这里不会出什么乱子。

这是个单一的过程，但却需要两只手操作，同时还有一个手法问题比较讲究。什么手法呢？那就是不允许同时按下 K_{GA} 和 K_{RA}。当按下 K_{GA} 时，数据还没有稳定下来，要是 RA 在这个当口工作，它保存的数据就很有可能是错的。

装载过程已经结束，第一个数“10”已经位于寄存器 RA 中了。现在，我们要用第二个数与它相加。这需要再次扳动那排开关，得到第二个数“5”。然后，按住 K_{GA} 不要松开，使“5”进入加法器的另一个输入端。加法器是自动即时相加的，它会立即计算出相加的结果“15”。此时，按一下 K_{TR} 将其保存到临时寄存器 TR 中，然后松开 K_{GA}。

因为是要做一连串的加法，所以当前的计算结果还必须参与下一次计算，这意味着要把数据从临时寄存器 TR 移动到 RA。

通常情况下，传输门 GA 是断开的，所以不用担心数据冲突，直接按住 K_{GB} 不要松手，使计算结果从寄存器 TR 通过 GB 流向 RA；接着，按一下 K_{RA} 将数据锁存，最后将 K_{GB} 松开。

有趣的是，一旦数据从传输门 GB 流出来，它不但会等待 RA 将其保存，同时也流向加法器的另一个输入端，并和 RA 中原有的数相加。不过不用担心，寄存器 TR 会将结果拦住，以防止形成一个反馈。

很明显，在这道加法题中，除了第一个数字“10”需要预先保存到寄存器 RA 之外，从第二个数字“5”开始，一直到最后一个数字“6”，所有数字在相加时的操作过程都是一样的，都要经历用开关扳数、相加并保存到寄存器 TR，然后从 TR 移动到 RA 的过程，这个过程可以简单地称为“相加”。当最后一个数加完之后，最终的结果仍然在寄存器 RA 中。

当然，寄存器 RA 只有 5 位，能表示的最大数是 11111，也就是十进制的 31，如果我们要加起来的数很多，而且每个数都很大的话，它可能会产生一个进位，但这个进位将被加法器丢掉，这将在 RA 中得到不正确的结果。没关系的，你看，在这道运算题里，数字又小又少，就是保证它正常运行即可。我们的目标是先让它能工作起来，再想办法让它完善。

10.3 简化操作过程

用我们发明的计算器做一连串的加法，比我们原先所想的要麻烦多了。不过话又说回来，这似乎也很有趣。

有趣归有趣（当然，时间一长你就不一定还这样想了），想必你也希望操作越简单越好。要不然的话，一手一只开关，左右开弓，还真是挺别扭，有时候不注意还会搞错，本来是要按这个，却按了那个。既然我们的观点一致，那就开始搞一搞技术革新吧。

注意，我的真实想法是去掉那些操作开关（左边那排用来输入数据的开关还得留着，没有它们我们只能计算空气），用别的方法来完成这些操作。

开关的作用是接通或者断开电源，使电路从 0 变成 1，或者从 1 变成 0。开关可以做到的事，逻辑电路也能做到。如图 10.9 所示，我们重新发明了一个新的逻辑电路，这是个新生事物，一时间我还不知道该给它起个什么名字，姑且就叫“我们的新电路”吧。该电路有 4 个输出，用于代替以前的那些开关。

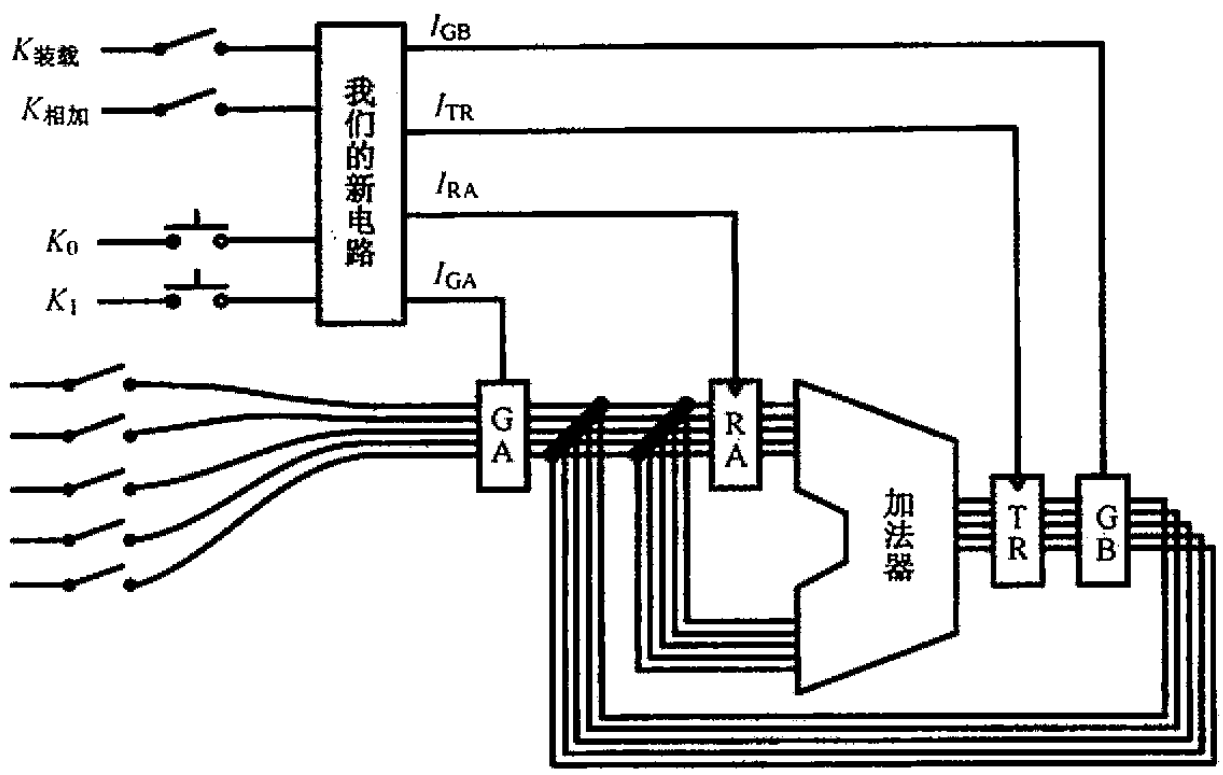

图 10.9　一个使相加的操作过程变得有规律的电路设计

一个普通的逻辑电路而已，它不可能精灵古怪到能自动按正确的时间和顺序产生输出。现在不是做梦的时候，我们所能做的，就是给它提供相应的输入。

设计逻辑电路是门学问，要是你掌握了它，就会知道，要得到相同的输出，可以有很多不同的设计方法。在这里，我们用 4 个开关作为输入。表面上看，我们只是给原来的开关换了位置，拆了东墙补西墙，换汤不换药，赶走了 4 个无赖，却又不得不请来 4 个讨人嫌的家伙帮忙。不要误会，这只是表面现象，很快你就会发现它的妙处。

由于刚刚做过加法，我们知道，要把一大堆数加起来，首先要执行一个“装载”动作，后面都是清一色的“相加”。这两件事情，一个是盛饭，一个是不停地吃，虽然有关联，但毕竟是两码事。所以，我们用两个开关 $K_{装载}$和 $K_{相加}$来指明要做哪件事。合上 $K_{装载}$，断开 $K_{相加}$，就是要装载一个数到寄存器 RA 中；断开 $K_{装载}$，合上 $K_{相加}$，就表明我们要开始做加法了，相信这很好理解。

还有，差点忘了提醒你注意这两只开关的类型，它们都是铡刀开关，断开和闭合都需要分别动一次手。采用这种类型的开关，是因为它们在后面的操作中基本上不需要扳来扳去。

一旦 $K_{装载}$和 $K_{相加}$被扳到合适的状态，就意味着大政方针已定，剩下的事情就是分步骤来完成这个目标。逻辑电路不是精灵，不改变它的输入，它就不会有相应的输出，而 K_0和 K_1的作用就在于此。当我们按顺序分别按下 K_0和 K_1时，就可以完成“装载”或者“相加”的全过程。我们会喜欢这种操作方式的，因为它有规律可循，只需要按顺序分别按动两个开关即可，你还能找到比这更不用动脑子的事情吗？

注意，与 $K_{装载}$和 $K_{相加}$不同，K_0和 K_1是按键开关，按下去之后，一松手，它

还会自动弹开。之所以采用这种开关，是因为它们需要频繁操作，我们不想受累。记得大明星陈慧琳唱过一首歌，叫《不如跳舞》，里面有一句唱得好："让自己觉得舒服，是每个人的天赋。"

不过，为了完成"装载"或者"相加"，只用两个开关 K_0 和 K_1 够吗？答案是，不多不少，刚刚好。为了证明这一点，同时也为了设计这个逻辑电路，我们需要详细地定义一下它的工作状态，设计一张表格，并从中得到逻辑表达式。

首先，不管 $K_{装载}$ 和 $K_{相加}$ 的状态如何，只要 K_0 和 K_1 中的任何一个没有接通，所有的输出都必须是 0，如图 10.1 所示，这可以禁止错误的操作。

表 10.1 K_0 和 K_1 未按下时的状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0

不过，同样是 $K_{装载}$ 闭合、$K_{相加}$ 断开的情况下（这表明我们要开始干装载的活了），如果按下了 K_0，则在逻辑电路的输出端，只有 I_{GA} 和 I_{RA} 为 1，传输门 GA 打开，寄存器 RA 执行锁存动作，保存从左边那排开关上来的数据，如表 10.2 所示。

表 10.2 执行装载功能、当 K_0 按下时的状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
1	0	1	0	1	1	0	0

这里有个问题，寄存器 RA 必须等待从 GA 来的数据稳定下来才能动作。换句话说，I_{RA} 应该比 I_{GA} 晚一点为"1"才行。在足球场上，守门员必须睁大眼睛，看看球从哪个角度过来才能采取行动，要是人家刚准备射门，与此同时他就来个饿虎扑食，这未免也太心急了些。

那为什么不换一种方法，按下 K_0 的时候，I_{GA} 先为"1"；按下 K_1 的时候，I_{RA} 再为"1"呢？

原因很容易理解，逻辑电路的输入直接对应着输出，要想在两个不同的输入之间保持某个输出的连贯性，这既困难，也不合理。

这确实是个问题，但眼下还没有办法解决，只能先放一放。

可以看出，要做"装载"的工作，只需要按一下 K_0 就行，K_1 是多余的，它存在的原因完全是因为后面的相加过程需要两个步骤，必须使用两个开关。既然是

这样，我们强迫译码电路在 K_1 按下的时候什么也不做，如表 10.3 所示。

表 10.3　执行装载功能、当 K_1 按下时的状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
1	0	0	1	0	0	0	0

“装载”的过程已经完成，接下来是干“相加”的活了，也就是将另一个数和寄存器 RA 中的数相加，这需要闭合 $K_{相加}$，断开 $K_{装载}$。此时，如果按一下 K_0，则 $I_{GA} = I_{TR} = 1$，数据通过 GA 进入加法器，与寄存器 RA 中的数相加之后锁进临时寄存器 TR，如表 10.4 所示。

表 10.4　执行相加功能、当 K_0 按下时的状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
0	1	1	0	1	0	1	0

这里同样有一个脉冲先后的问题，还是先不管它。

一个完整的相加过程还包括把结果返回寄存器 RA 的动作。这需要接着按一下 K_1，使 $I_{GB} = I_{RA} = 1$，数据离开临时寄存器 TR，穿过传输门 GB，到达寄存器 RA 后被锁存，如表 10.5 所示。

表 10.5　执行相加功能、当 K_1 按下时的状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
0	1	0	1	0	1	0	1

这又是一个脉冲先后的问题，但是不要担心，天是塌不下来的，我们马上就会把它解决掉。

在这个电路上工作，重要的是必须按顺序按下 K_0 和 K_1，一个按下的时候，另一个已经自动弹开。这意味着，无论 $K_{装载}$ 和 $K_{相加}$ 的状态如何，只要是 K_0 和 K_1 同时按下的情况，逻辑电路的输出就一律为“0”，如表 10.6 所示。

表 10.6　当 K_0 和 K_1 都按下时的状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
0	0	1	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	1	1	0	0	0	0
1	1	1	1	0	0	0	0

不过，即使 K_0 和 K_1 只有一个为“1”，如果 $K_{装载}$ 和 $K_{相加}$ 都断开，或者都闭合，

这也不是我们所希望的，所以一律将电路的所有输出都置“0”，如表 10.7 所示。

表 10.7　其他一些无效的开关状态

$K_{装载}$	$K_{相加}$	K_0	K_1	I_{GA}	I_{RA}	I_{TR}	I_{GB}
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
1	1	0	1	0	0	0	0
1	1	1	0	0	0	0	0

4 个开关，每个开关都有“0”和“1”两种状态，这样，它们就会有 16 种组合，已经分别列于前面 7 个表中。俗话说，拿着锤子三年，看什么都像钉子。我怀疑有些人整天画真值表，研究如何从这些表中得到逻辑表达式，时间长了，就是看到渔网都有写逻辑表达式的冲动。我相信你不至于这样痴迷，但是最起码的，如何从这张大的真值表中得到 I_{GA}、I_{RA}、I_{TR}、I_{GB} 的逻辑表达式，并根据这些逻辑表达式来组装这个“我们的新电路”，对你来说应该不会是个问题，对吧？因为我相信你已经从亚里士多德、布尔和香农那里得到了启示。更何况，我们曾经做过“大型机电设备”的报警器，发明了全加器，有这些辉煌的历史，眼前这个电路还不是小菜一碟吗？

10.4　这就是传说中的控制器

理想中，在这台经过全副武装的机器上做加法是很有趣的，因为它不但简化了操作，而且整个操作过程还具有很强的规律性——在我看来，它实在是太有美感了。还是前面那道数学题：

$$10+5+7+2+6$$

回到图 10.9，要计算上面那些数字的总和，首先合上 $K_{装载}$，断开 $K_{相加}$，用左边那排开关扳出数字“10”，然后分别按一下 K_0 和 K_1。这时，“10”就被保存到寄存器 RA 里了。

接着，断开 $K_{装载}$，合上 $K_{相加}$，用左边那排开关扳出数字“5”，再分别按一下 K_0 和 K_1，于是我们就得到了相加的结果“15”，它位于寄存器 RA 中。

不要再理会 $K_{装载}$ 和 $K_{相加}$，让它们维持现状，毕竟后面全是在做加法。现在，再用左边那排开关扳出第三个数“7”，再次分别按下 K_0 和 K_1，于是又会在寄存器 RA 中得到本次相加的结果“22”。

按顺序，后面要加的数是“2”和“6”。但是，不管后面还有多少数字要加，操作过程都一模一样。

理想毕竟只是理想，正如前面已经指出的那样，这个电路实际上是有缺陷的。

比如，它总是让 I_{GB} 和 I_{RA} 同时为“1”，以便把相加的结果从寄存器 TR 移动（复制）到 RA。但是，我们知道，后者必须比前者晚一点为“1”才能保证可靠性。

要让“我们的新电路”正常工作，继续完善它是不可避免的。有趣的是，改造后的电路不但工作起来可靠，同时还能获得一个额外的好处，那就是它现在只用三个开关，比以前少了一个——呃，我知道你不在乎多一个开关的钱，但事实上，这不是少一个开关的问题，最重要的，操作起来简直是前所未有地简单！

K_0 和 K_1 的作用不过是产生脉冲。在我们的操作下，先是 $K_0=1$，然后 $K_1=1$，接着又是 $K_0=1$，……这很容易让我们想起循环移位寄存器来。既然移位寄存器也能达到相同的效果，那就没有理由不在我们的电路中使用。

为了取代 K_0 和 K_1，需要用两个上升沿 D 触发器来制造一个循环移位寄存器，并把它加进我们的电路中，如图 10.10 所示。

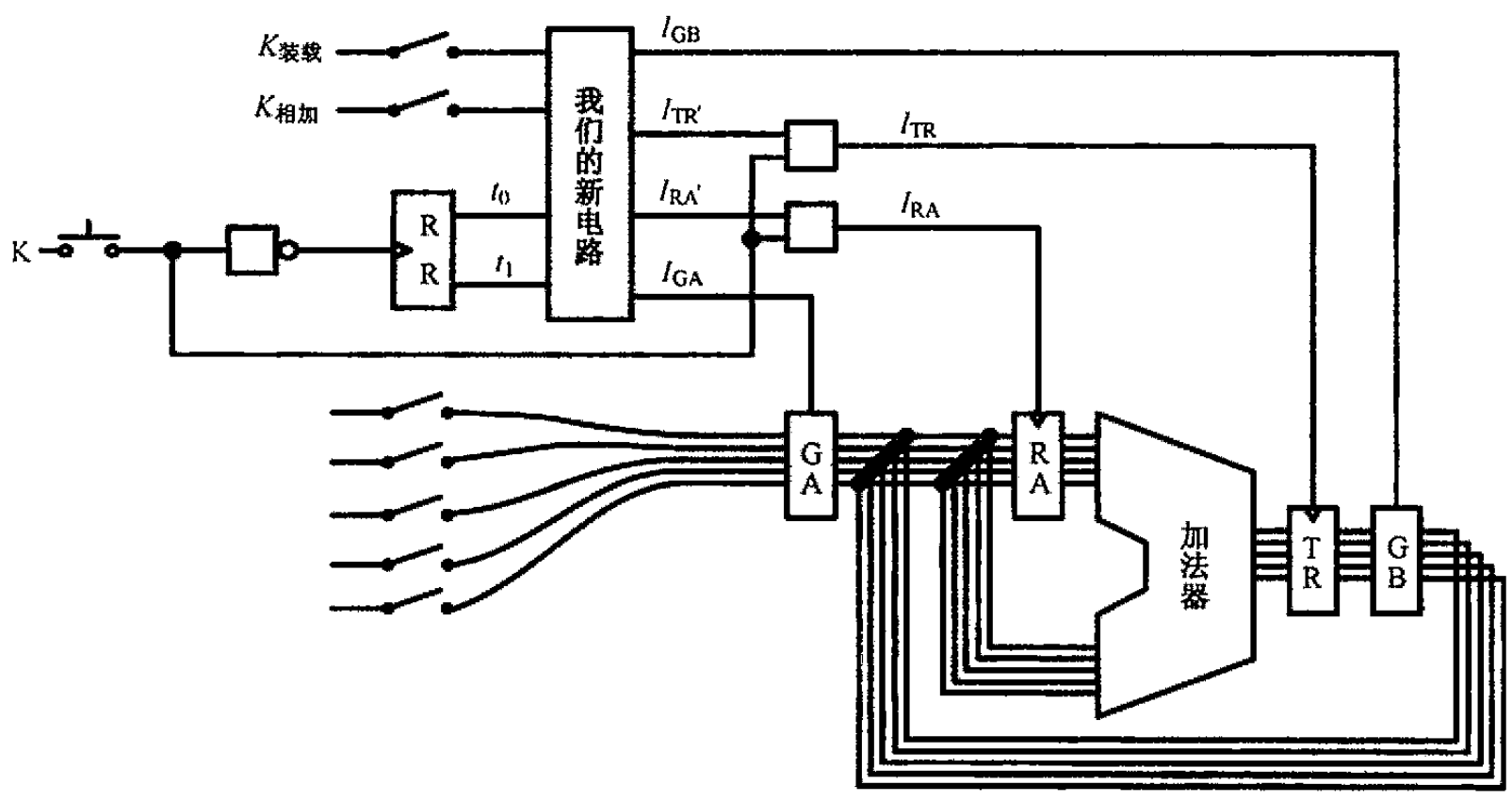

图 10.10　使用循环移位寄存器来简化装载和相加过程

图中，RR 就是我们刚刚加进来的循环移位寄存器，它有两个输出 t_0 和 t_1，就是这两个家伙取代了原先的开关 K_0 和 K_1，让它们失了业。

另一个比较大的改动是电路的输出部分。在这个新的方案里，$I_{RA'}$ 和 $I_{TR'}$ 其实就是以前的 I_{RA} 和 I_{TR}，我们重新定义了 I_{RA} 和 I_{TR}，现在，它们分别来自两个与门的输出。

现在，我们来分析一下这个新的设计是如何运作的。开始之前，应当先设置循环移位寄存器 RR 的初始状态，使它的两个输出 t_0 和 t_1 分别为 1 和 0。

现在要开始计算一连串的加法了。首先是装载第一个数到寄存器 RA，这需要用左边那排开关来扳出这个数字。

接着，闭合 $K_{装载}$，断开 $K_{相加}$，此时，由于 RR 的输出 $t_0=1$，按照以前的设计，

$I_{GA} = I_{RA'} = 1$，传输门 GA 打开，如图 10.11 所示。

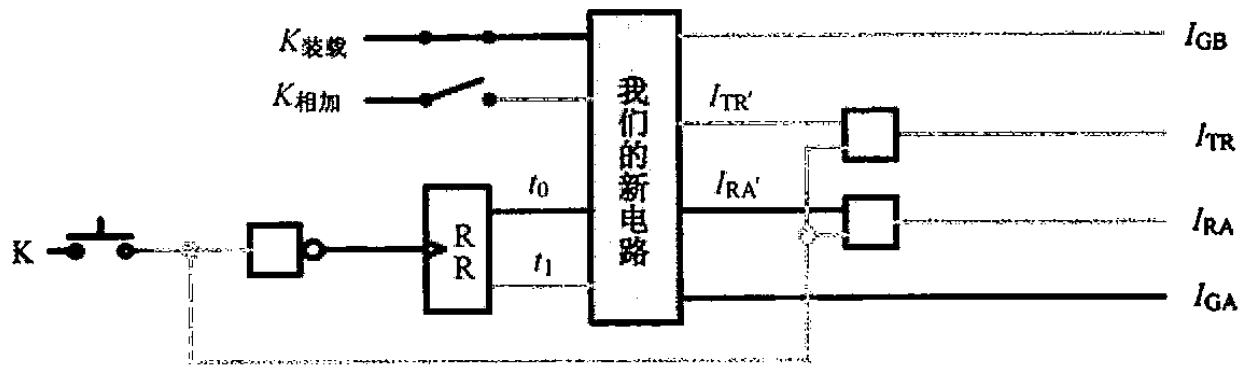

图 10.11　装载数据的过程（1）

现在数据已经送到寄存器 RA 的嘴边了，但它还吃不了，因为 K 没有按下，所以通过与门输出的 $I_{RA} = 0$。这是有意的，我们说过，寄存器 RA 需要等数据稳定下来之后才能动作。

现在，我们按下开关 K。如图 10.12 所示，随着 K 的接通，将同时产生两路脉冲，第一路通过非门到达循环移位寄存器 RR。遗憾的是，这是一个由高到低的下降沿，RR 不会理睬。这也意味着，“我们的新电路”仍然保持原来的输出纹丝不动。

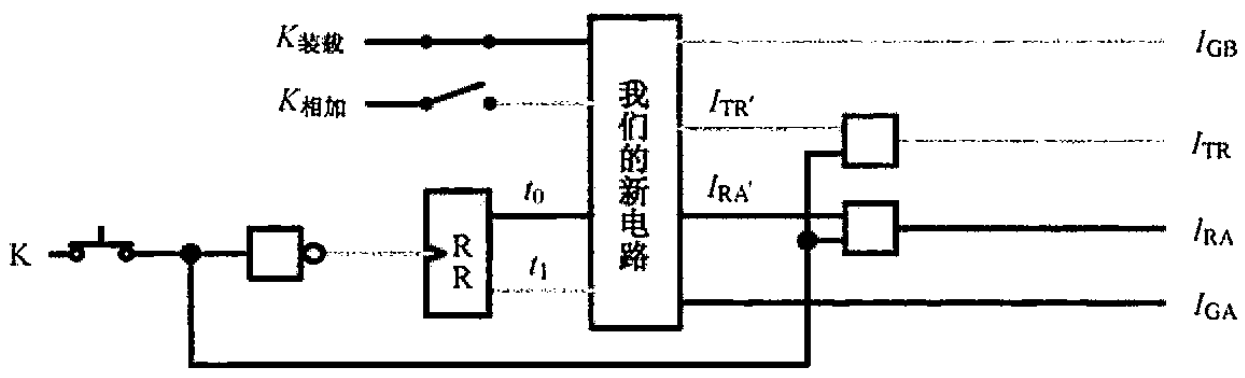

图 10.12　装载数据的过程（2）

与此同时，另一路脉冲被直接送到与 I_{RA} 相连的与门，使得 I_{RA} 从 0 变到 1。在它的上升沿，寄存器 RA 将数据锁存。哇，这个过程真是太精彩了，不是吗？

很显然，我们用心良苦，搞得如此复杂，只是希望在 K 闭合和松开的过程中，I_{RA} 比 I_{GA} 慢半拍出现，好安全地将数据锁住。现在，隐患已经消除了。

上面说的，是当 K 按下时所发生的事情。看得出来，电路构造绝对巧妙，而它的工作过程也绝对精彩。不过，我们不能一直按着它，毕竟还有很多事要做。

因为是按键开关，当我们一撒手，K 马上又松开了。如图 10.13 所示，按键松开的瞬间，它产生的“0”经非门后变成“1”。这个变化对于循环移位寄存器 RR 来说是个不折不扣的上升沿。也许它等待这一刻已经很久了，于是循环移动一次，$t_0 = 0$ 而 $t_1 = 1$，根据前面的设计，所有的输出都是 0，自动进入下一个步骤。

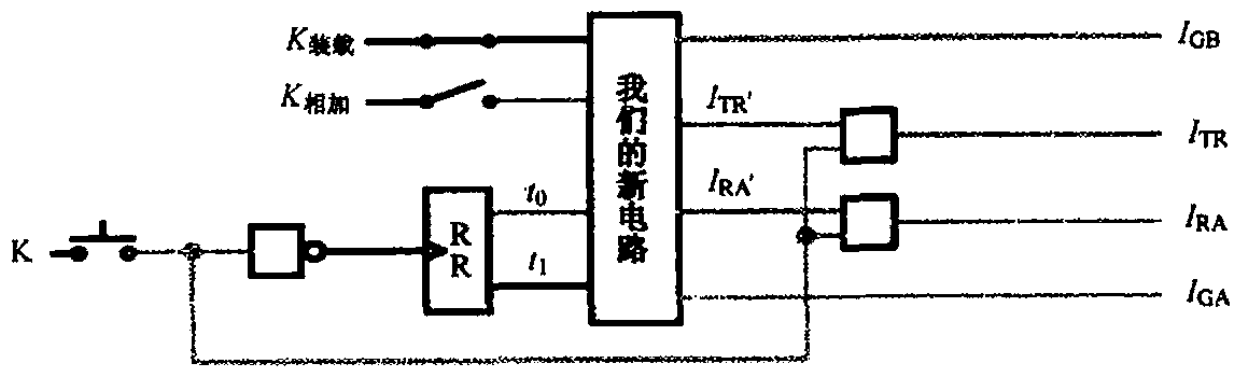

图 10.13　装载数据的过程（3）

尽管完成“装载”的工作只需按一次开关 K 就可以，但是在前面设计这个电路的时候，我们预设的是做一件事需要按两次，已经按了一次，现在再按一次。

如图 10.14 所示，因为所有的输出都是“0”，所以即使 K 被按下，它们也不可能突然就变成“1”。同时，K 按下时，对 RR 来说是下降沿，这不会对它有任何触动——换句话说，它对此无动于衷。

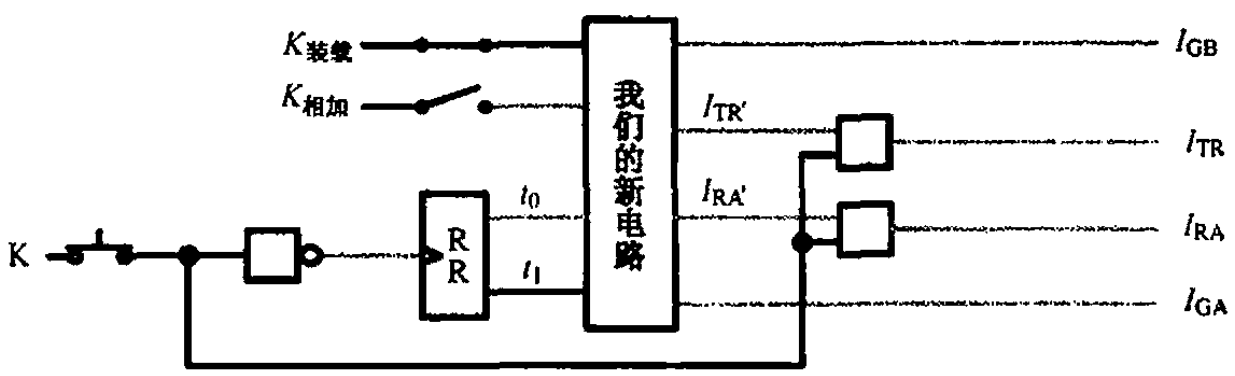

图 10.14　装载数据的过程（4）

但是，和前面一样，当 K 松开时，RR 又循环移动一次，使得 $t_0=1$、$t_1=0$，这又回到了一开始，也就是图 10.11 所示的状态。这意味着，你可以再来一次“装载”的过程。

现在，你有两个选择：第一，要是你觉得刚才装载的数据不对，想重新装载一次，可以直接再按两次 K；第二，如果你准备开始做加法，干“相加”的活儿，就断开 $K_{装载}$，合上 $K_{相加}$。

我相信你会选择第二个，也就是接着做加法。出于这个目的，我们再用那排开关准备好第二个要相加的数，并断开 $K_{装载}$，合上 $K_{相加}$。如图 10.15 所示。真奇怪，这个电路马上改变了状态，根据前面的设计，$I_{GA}=I_{TR'}=1$，传输门 GA 打开，提前使另一个数进入加法器并开始计算。

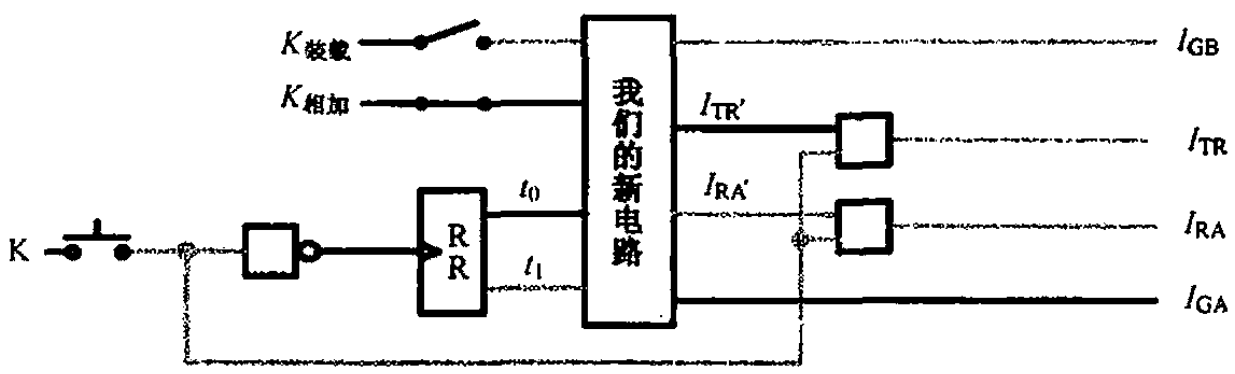

图 10.15　相加过程（1）

现在按一下 K 执行第一个动作，和前面一样，先是 $I_{TR'}$和从 K 来的上升沿脉冲一起，使得 I_{TR}从 0 翻转到 1，临时寄存器 TR 锁存相加的结果，如图 10.16 所示。

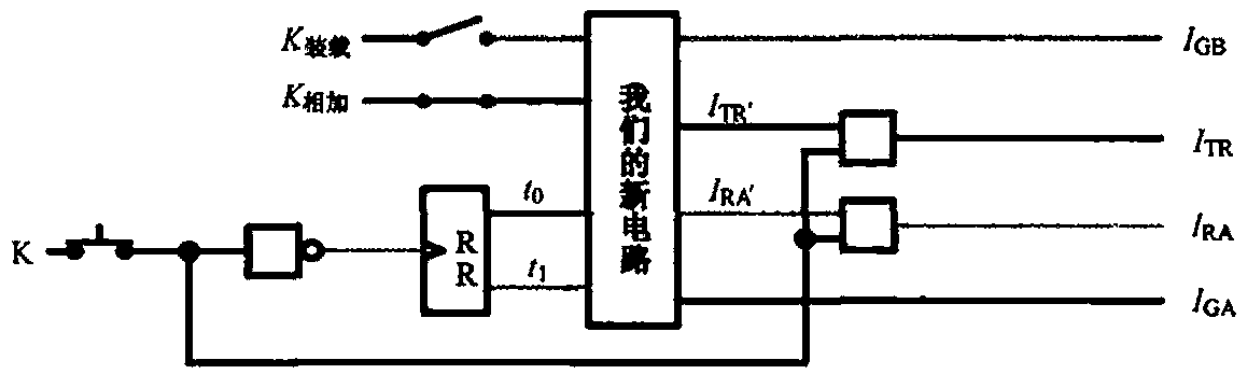

图 10.16　相加过程（2）

完整的“相加”过程需要两步，这是第一步，已经得出了相加的结果，现在需要把这个结果移动（复制）到寄存器 RA 中。好在我们的电路就是这样设计的。所以，当 K 松开时，RR 再次循环移动一次，$t_0=0$、$t_1=1$，于是 $I_{GB}=I_{RA'}=1$，为下一次 K 按下时将计算结果锁存到寄存器 RA 中提前做准备，如图 10.17 所示。

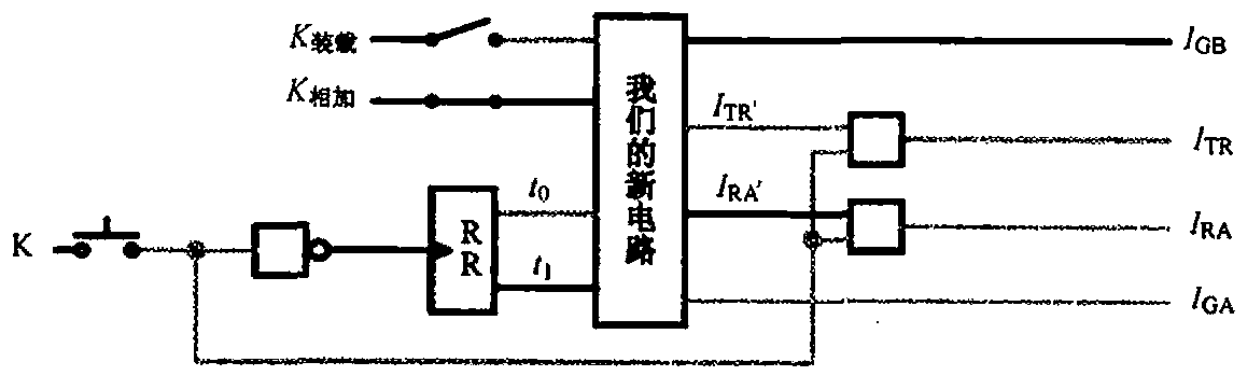

图 10.17　相加过程（3）

现在数据已经穿过传输门 GB，安全地来到寄存器 RA 的眼皮底下，只要再按一下 K，使得 $I_{RA'}$和从 K 来的上升沿脉冲一起，把 I_{RA} 从 0 翻转到 1 即可。在这个上升沿，寄存器 RA 锁存结果，如图 10.18 所示。

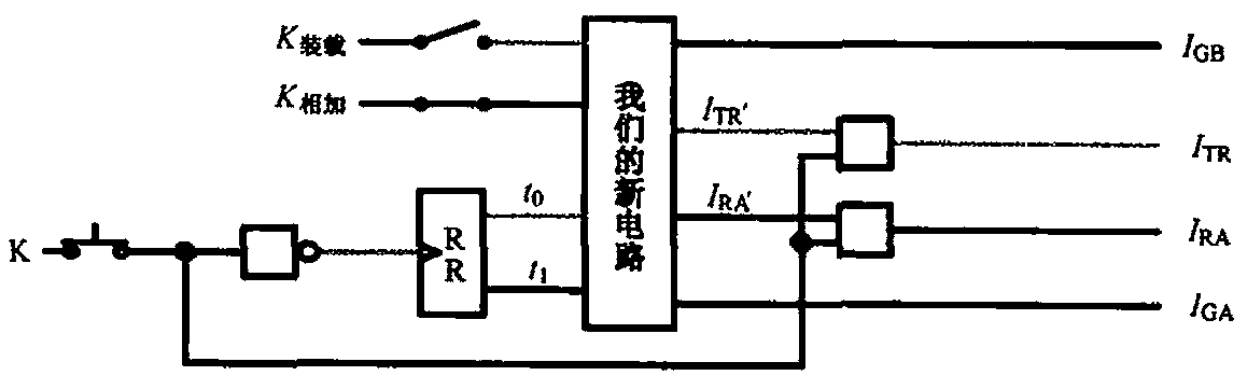

图 10.18　相加过程（4）

松开 K 时，循环移位寄存器从非门那里得到一个从 0 到 1 的上升沿脉冲，故再次循环移位，使得 $t_0=1$、$t_1=0$，于是整个电路又回到了“相加”过程的最开始，也就是前面的图 10.15 中。

这意味着什么？因为要做一连串的加法，除了最开始要将第一个数装载到寄存器 RA 之外，其他都是单纯的相加。所以，这意味着，从现在开始，你可以按照“用开关扳数——按两次 K——用开关扳数——按两次 K——……”这样的模式将所有的数都加完，最终的结果就在寄存器 RA 中。

经常地，我们听别人说每台计算机里都有一个控制器，它可以使整个计算机按照规定的步骤有条不紊地计算。事实上，我们现在讲的这个电路就是一个控制器。当然，它太简单——哎呀，简陋了，还需要进一步完善。不过这是以后的工作，我们的当务之急是解决另外一件同样很麻烦的事情。